

Introduction to Parallel Computing

Pacheco, Chapter 1

Moreno Marzolla
Dept. of Computer Science and Engineering (DISI)
University of Bologna

Copyright © 2013—2026
Moreno Marzolla, University of Bologna, Italy
<https://www.moreno.marzolla.name/teaching/CP/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- prof. Salvatore Orlando, Univ. Ca' Foscari di Venezia
<http://www.dsi.unive.it/~orlando/>
- prof. Mary Hall, University of Utah
<https://www.cs.utah.edu/~mhall/>
- Tim Mattson
- Charles Leiserson, *Performance Engineering of Software Systems*, MIT OpenCourseware
https://youtu.be/o7h_sYMk_oc?si=5PyFuM_6laZDC7CC

Parallel Computing

- Instructor
 - Moreno Marzolla
 - <https://www.moreno.marzolla.name/>
- Web page
 - <https://www.moreno.marzolla.name/teaching/CP/>
- Schedule
 - Thu 14:00–17:00 Ercolani 3
 - Fri 10:00–13:00 Ercolani 3
- Lectures will be recorded on a best-effort basis
- Office hours
 - After the lectures
 - Online (Teams); please [send e-mail](#)

What you will learn

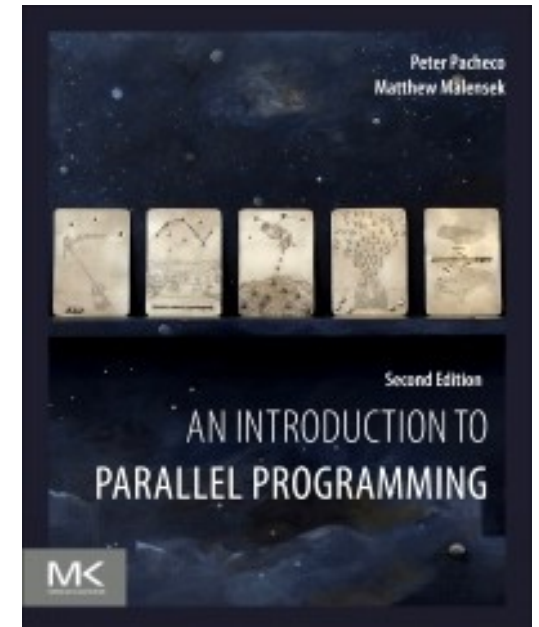
- What parallel computers are, and how they work
- How to design parallel programs
- How to program parallel computers
- How to evaluate the efficiency of parallel programs

Syllabus

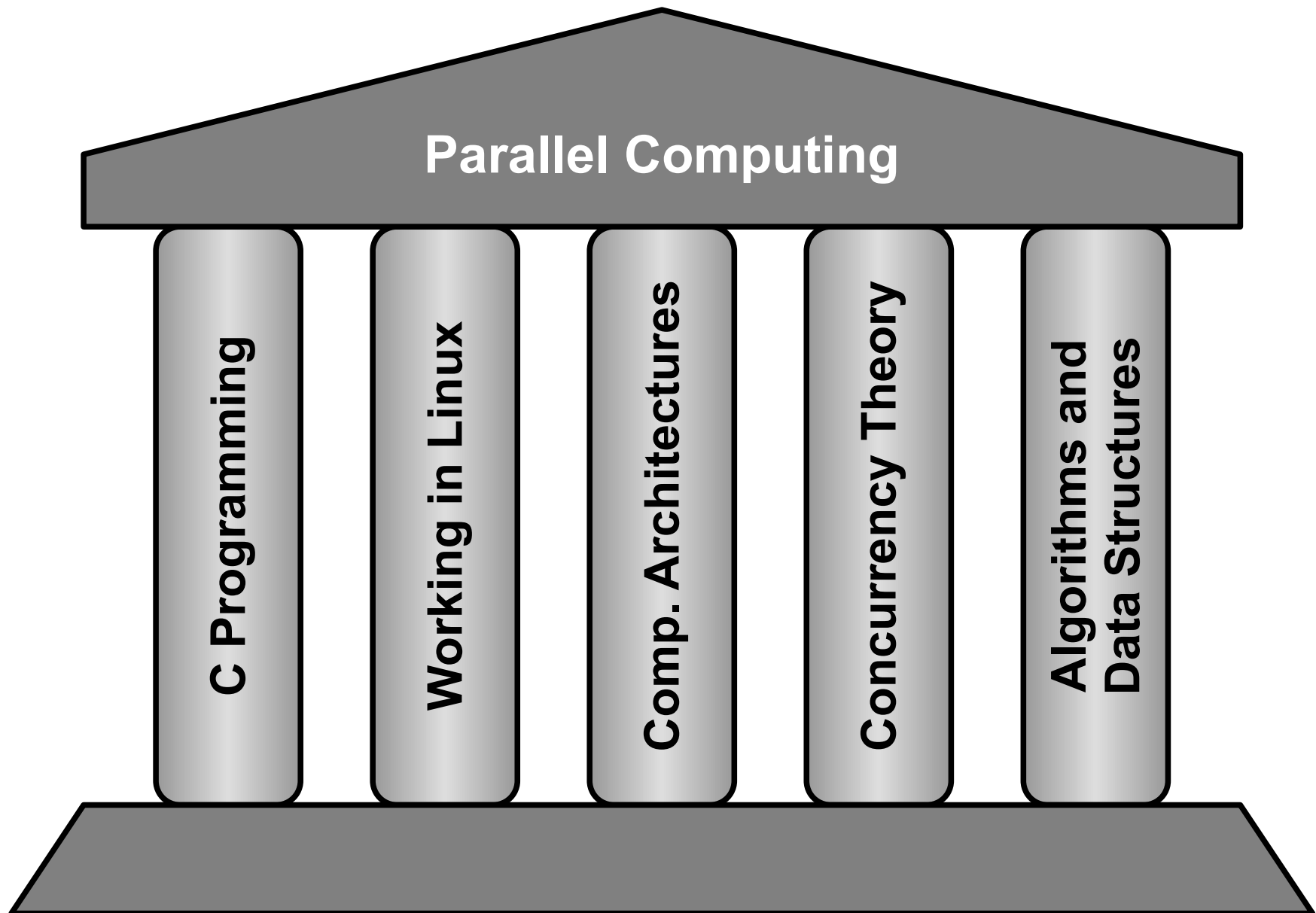
- 6 CFU
 - Lectures
 - Practical exercises (BYOD)
- Theory (first ~4 weeks)
 - Parallel architectures
 - Parallel programming patterns
 - Performance evaluation of parallel programs
- Programming parallel architectures (rest of the course)
 - Shared-memory programming with OpenMP
 - Distributed-memory programming with MPI
 - GPU programming with CUDA

Textbook

- Peter Pacheco, Matthew Malensek,
An Introduction to Parallel Programming, 2nd ed., Morgan Kaufmann, 2021, ISBN 9780128046050
 - The 1st edition is ok, too
- *CUDA C programming guide*
<http://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Slides and other material
<https://www.moreno.marzolla.name/teaching/CP/>



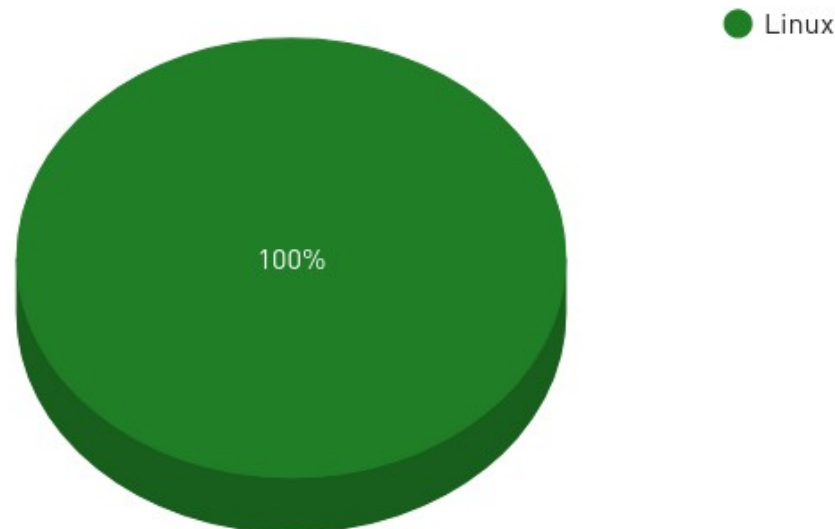
Prerequisites



Lab sessions

- Hands-on programming exercises, BYOD
- You only need a SSH client to connect to the department HPC Linux cluster
- Ask for an account following the instructions on *Virtuale*

Operating system Family System Share



Updated November 2025
Source: <http://www.top500.org>

Exam

- **Written exam** (weight: 60%)
 - In-person, closed book
 - Open questions and/or simple exercises on all topics discussed during the course
 - You can refuse the grade and retry the written exam
- **Individual programming project + written report** (weight: 40%)
 - Specification defined by the instructor; the same project for the whole academic year
 - Project can be handed in **at any time before Feb 28, 2027**
 - There is no discussion, unless I request it
 - **If you refuse the grade, you must hand-in a NEW project on NEW specifications**
- Final grade rounded to the nearest integer
- The written exam and programming project are **independent**, and **can be done in any order**

Generative AI policy

- The instructor did not use generative AI during the preparation of the teaching material
- **Generative AI shall not be used in any part of the final project** (code and report)
 - Spell-checking and other generic writing assistants are allowed



Questions?

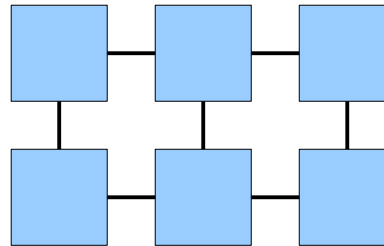
Intro to parallel programming

Parallel Computing

- Many applications need a lot of computing power
 - Weather forecast, climate modeling, physics simulation, product engineering, 3D animation, finance, ...
- Why?
 - To solve more complex problems
 - To solve the same problem in less time
 - To solve the same problem more accurately
 - To make better use of available computing resources

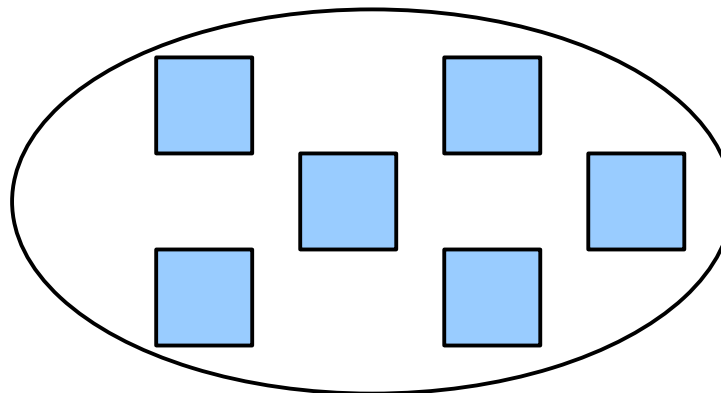
Parallel Computing (HPC) vs Cloud Computing (HTC)

- In **High Performance Computing (HPC)** we break applications into **strongly coupled** pieces executed on **specialized hardware** (CPUs and/or interconnection networks)



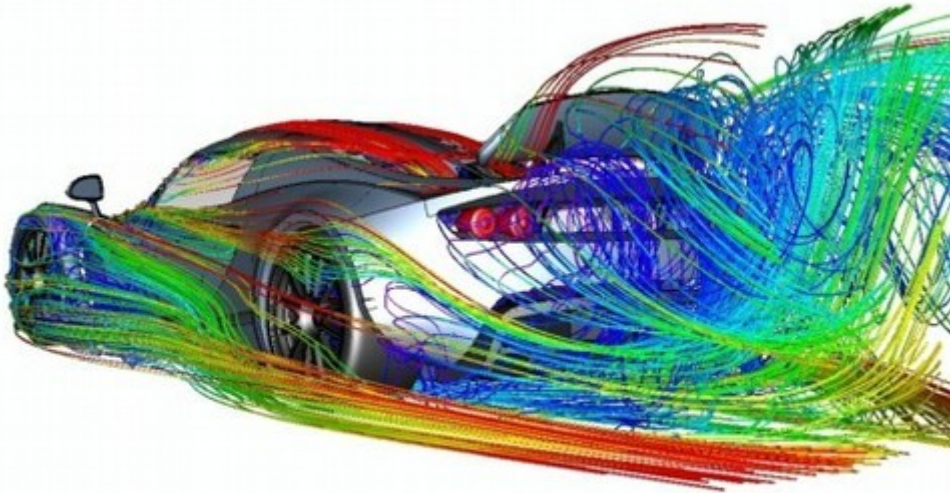
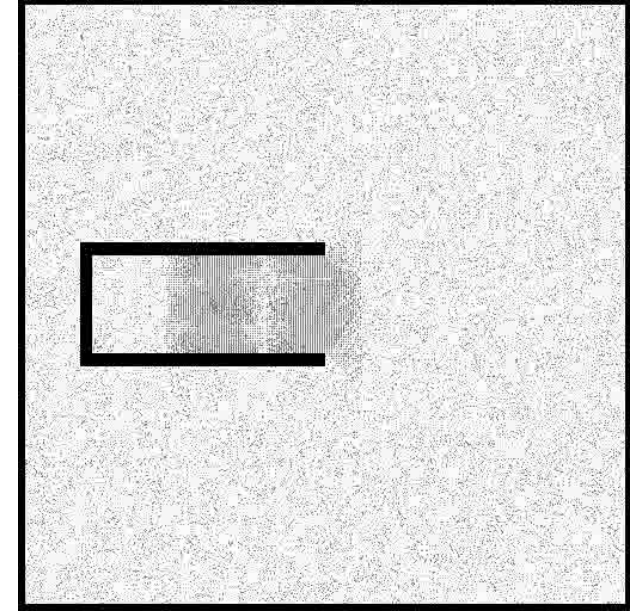
Interconnection network is important

- In **High Throughput Computing (HTC)** we break applications into **loosely coupled** pieces executed on **commodity hardware**

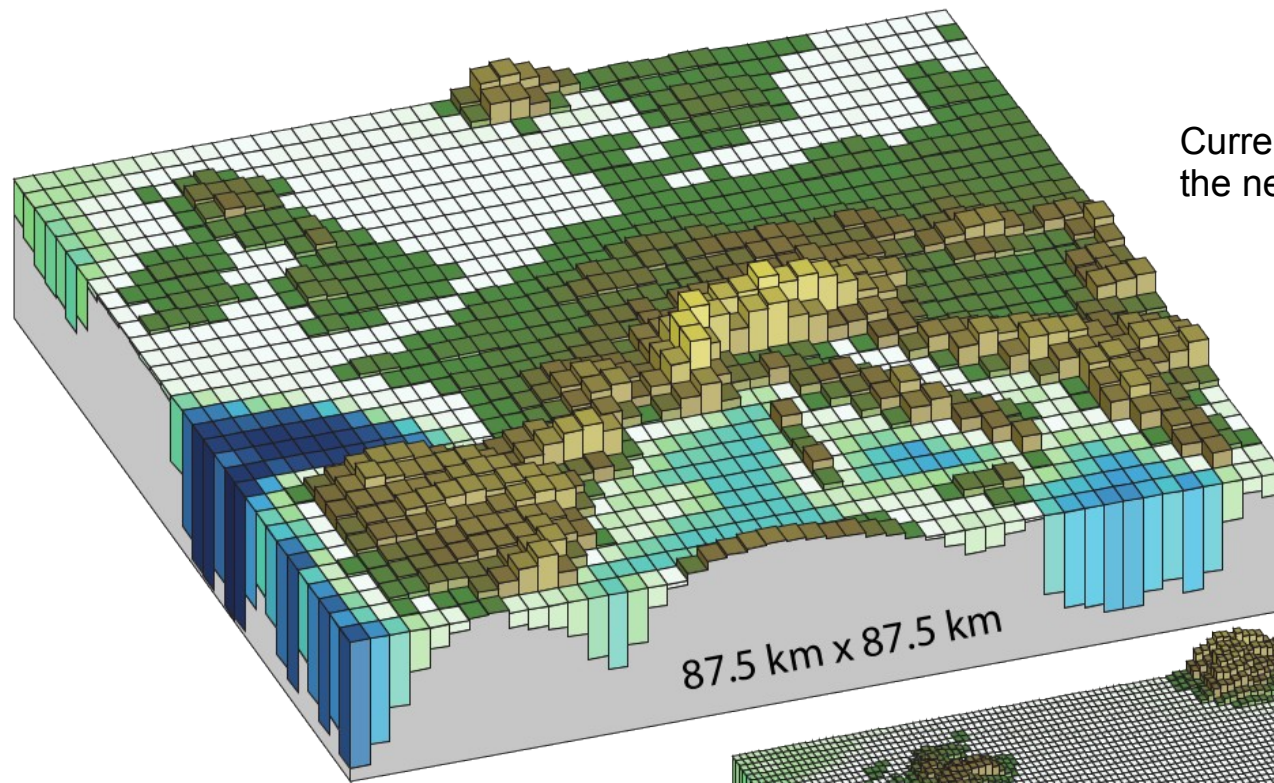


Interconnection network is less important

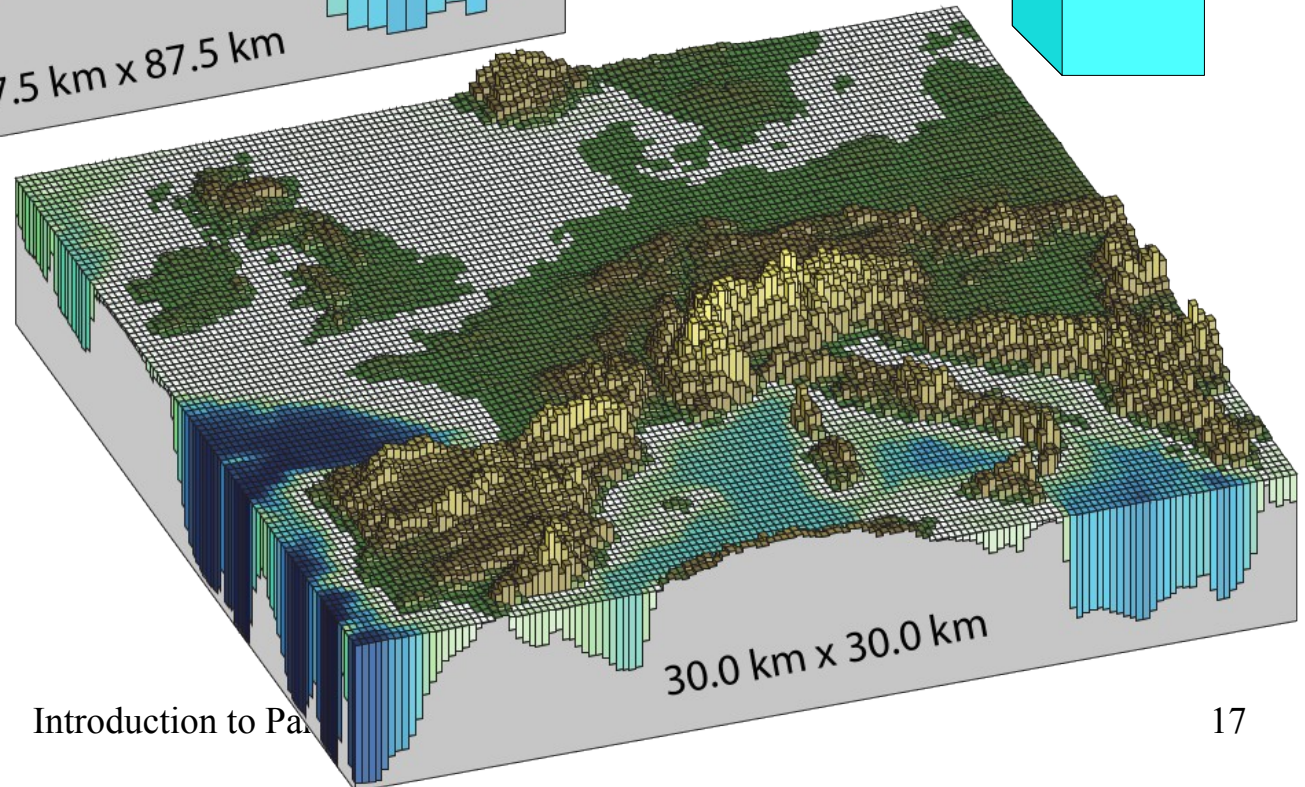
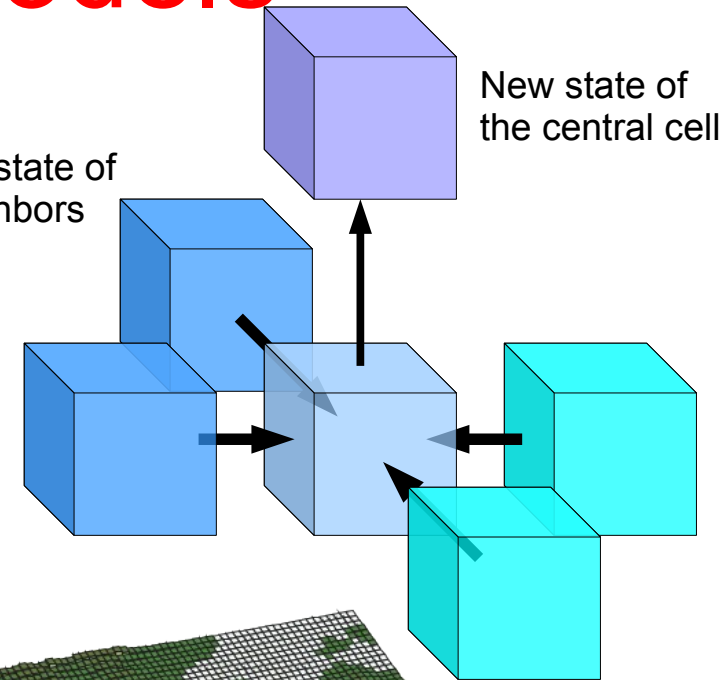
Numerical Wind Tunnel



Global Climate models

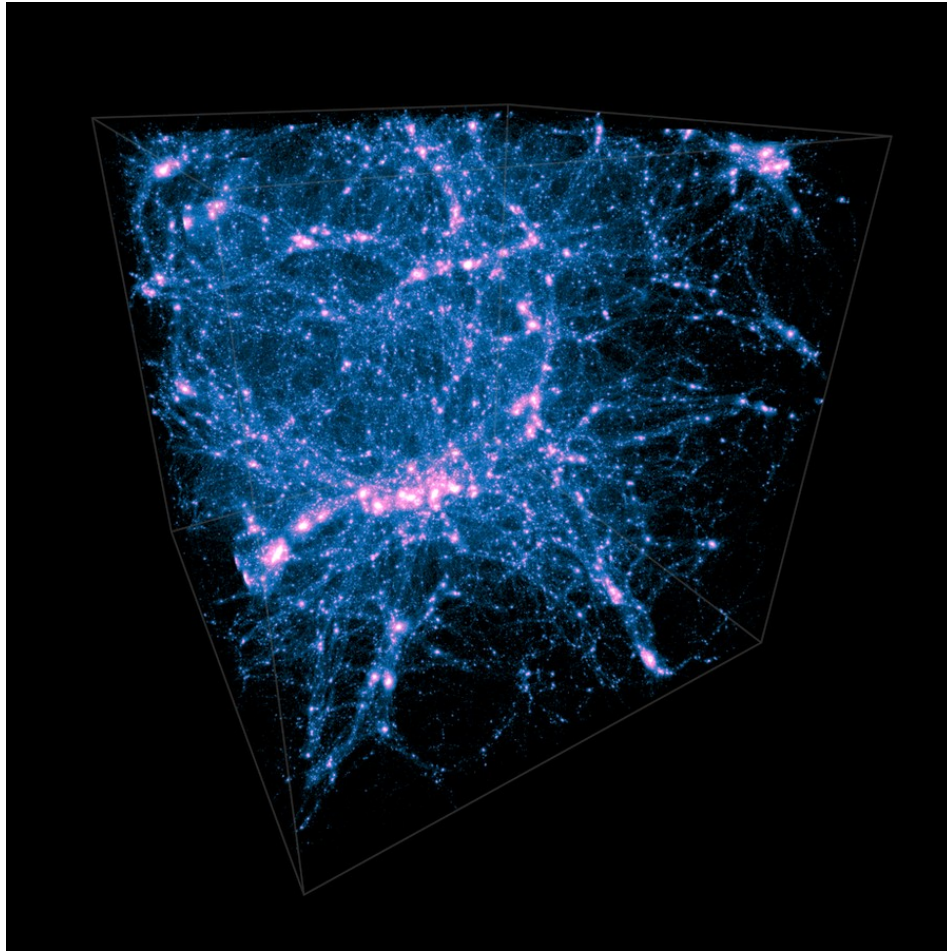


Current state of the neighbors



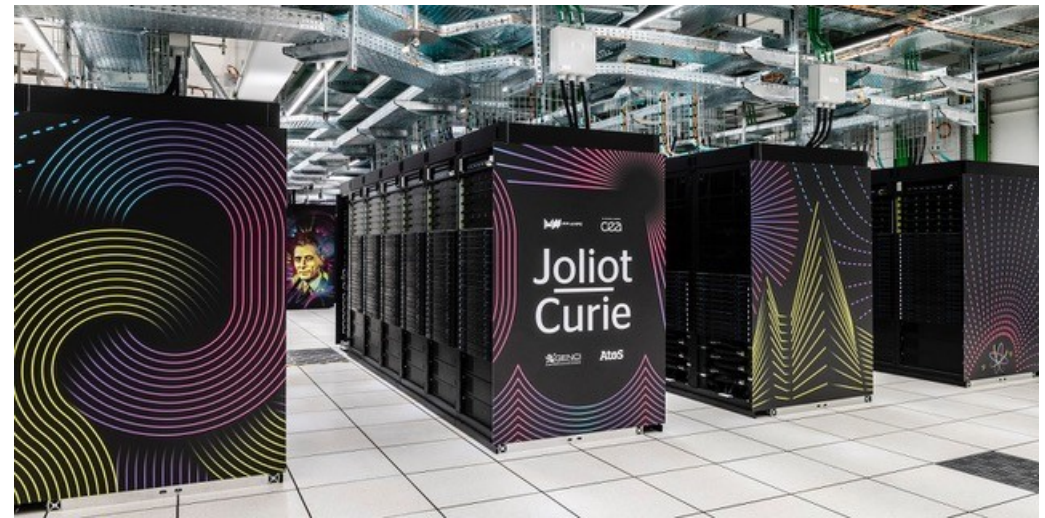
Source: Chapter 1 of *Climate Change 2013: The Physical Science Basis*
<https://www.ipcc.ch/report/ar5/wg1/>

Cosmological simulation: the Illustris project



“The Illustris simulations were run on supercomputers in France, Germany, and the US. The largest was run on 8,192 compute cores, and took 19 million CPU hours”

https://www.illustris-project.org/movies/illustris_movie_rot_sub_frame.mp4



Most of the Illustris simulation was executed in the Curie supercomputer with 440,000 x86 cores in nearly 5,000 nodes, 22 petaflop/s peak computing power

```
for each particle p {  
    compute the force on p by all other  
    particles p';  
    compute the new position of p using  
    Newton's second law;  
}
```

Moore's Law

"The number of transistors on an IC doubles every 24 months"

- That **used** to mean that every new generation of processors was based on smaller transistors

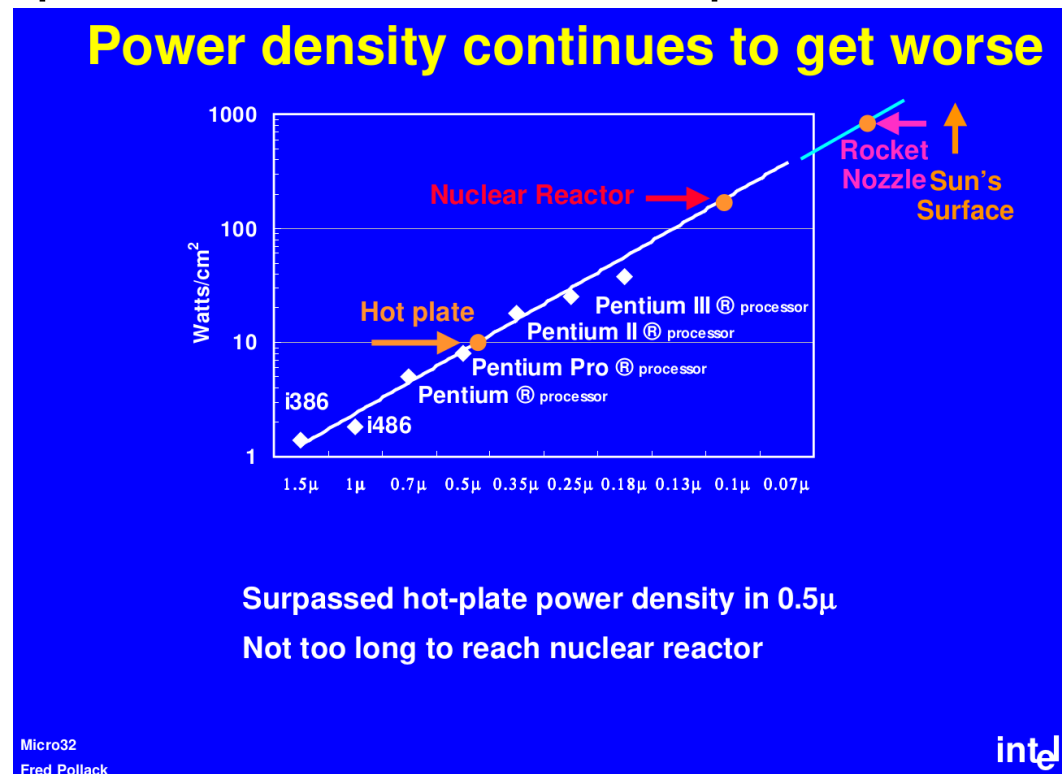
Moore, G.E., *Cramming more components onto integrated circuits*. Electronics, 38(8), April 1965



Gordon E.
Moore
(1929–)

Physics lesson

- Smaller transistors → Faster processor
- Faster processor → Higher power consumption
- Higher power consumption → More heat produced
- More heat produced → Unreliable processor



Power

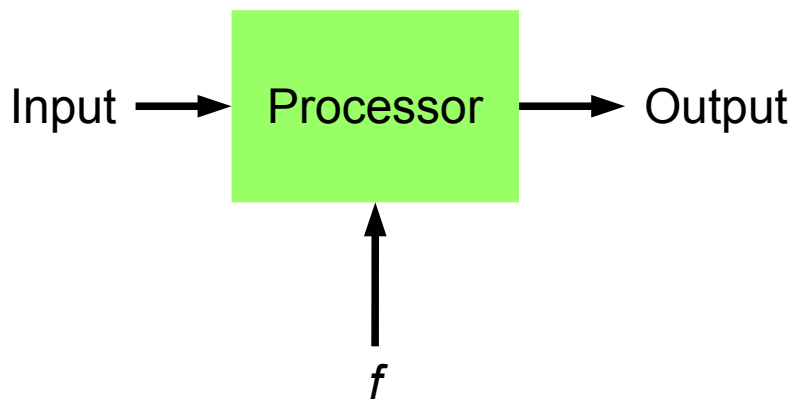
- The power required by an IC (e.g., a processor) can be expressed as

$$Power = C \times V^2 \times f$$

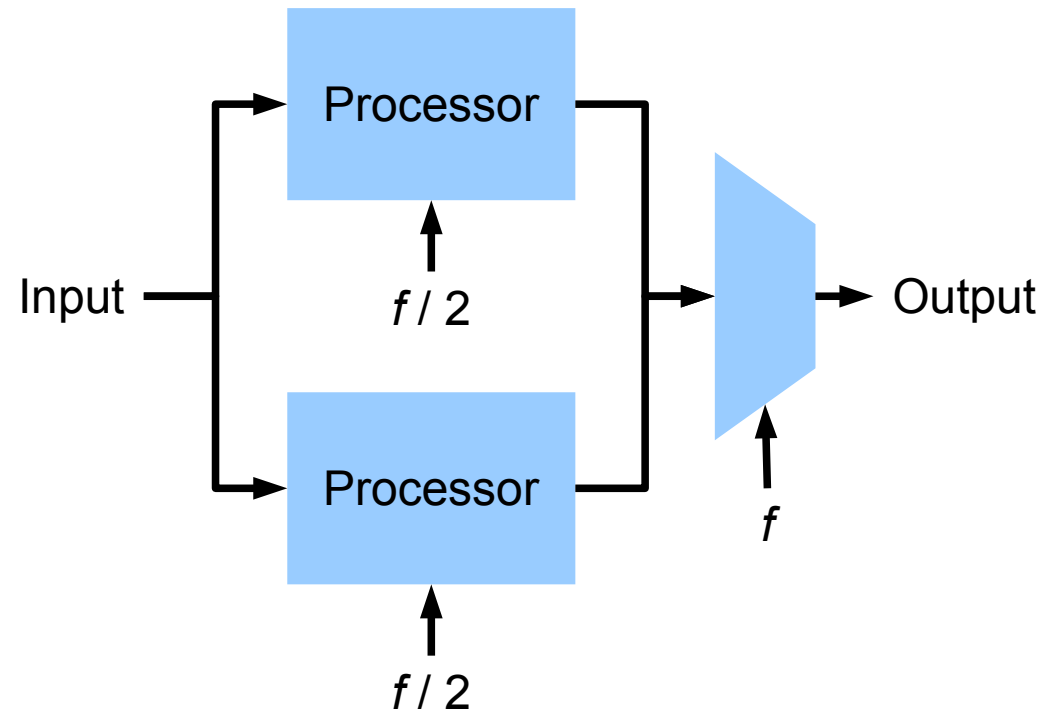
where:

- C is the *capacitance* (ability of a circuit to store energy)
- V is the *voltage*
- f is the *frequency* at which the processor operates

Power

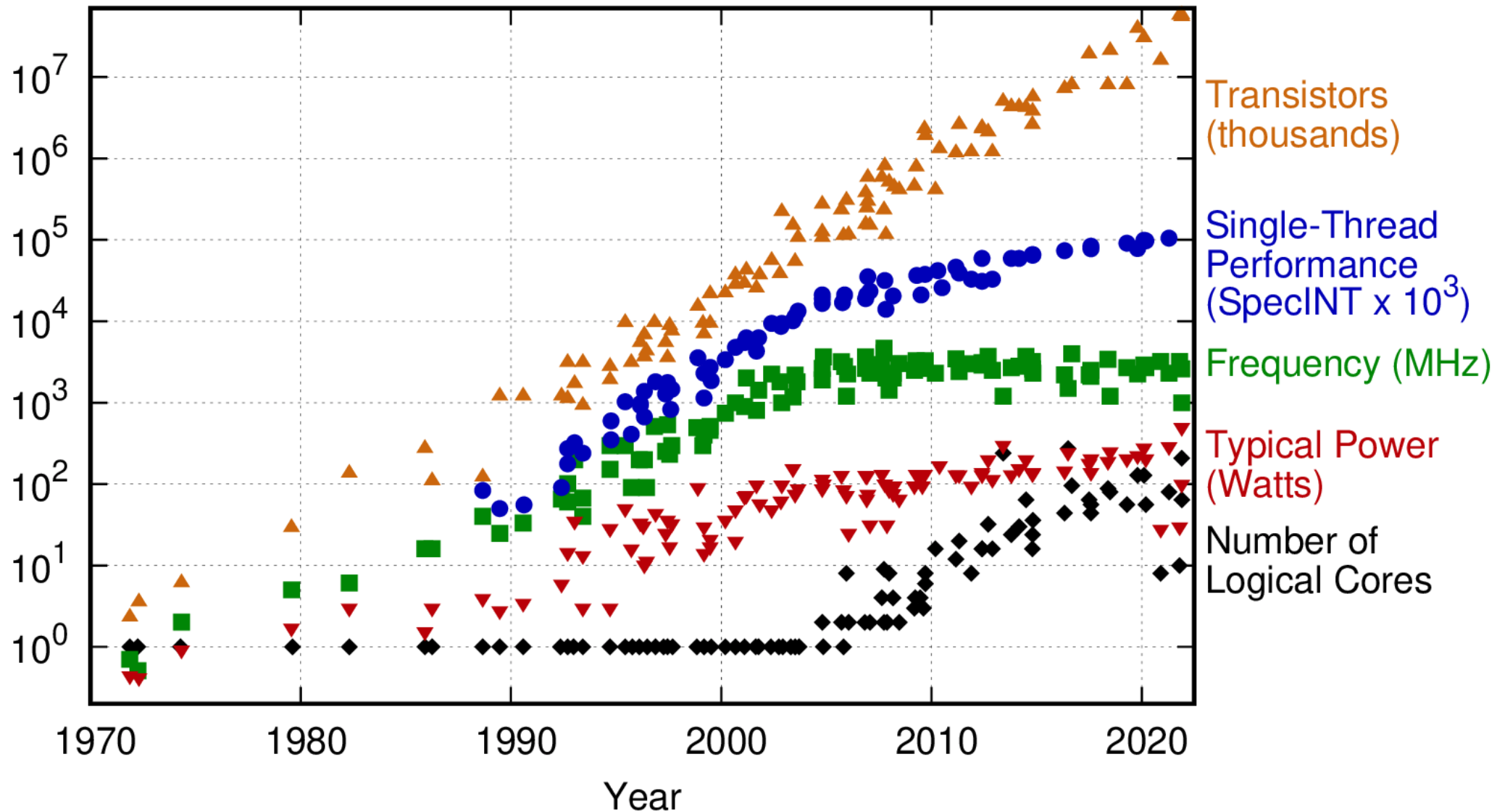


Capacitance C
Voltage V
Frequency f
Power = $C V^2 f$



Capacitance $2.2 C$
Voltage $0.6 V$
Frequency $0.5 f$
Power = $0.396 C V^2 f$

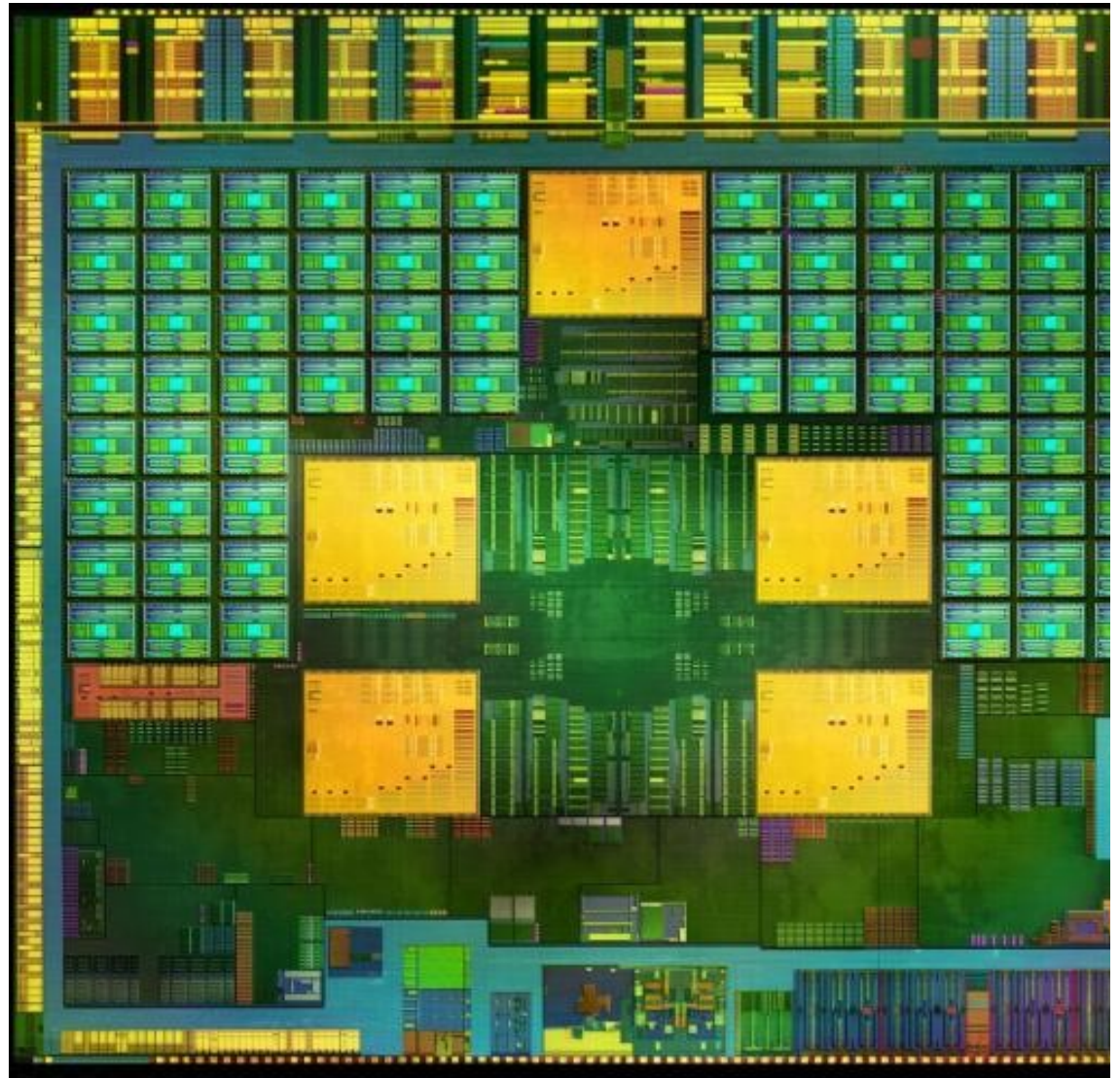
50 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

What happens today?

- HW designers create processors with more cores
- Result:
 - parallel hardware is **ubiquitous**
 - parallel software is **rare**
- The challenge
 - Make parallel software as common as parallel hardware



Parallel programming in brief

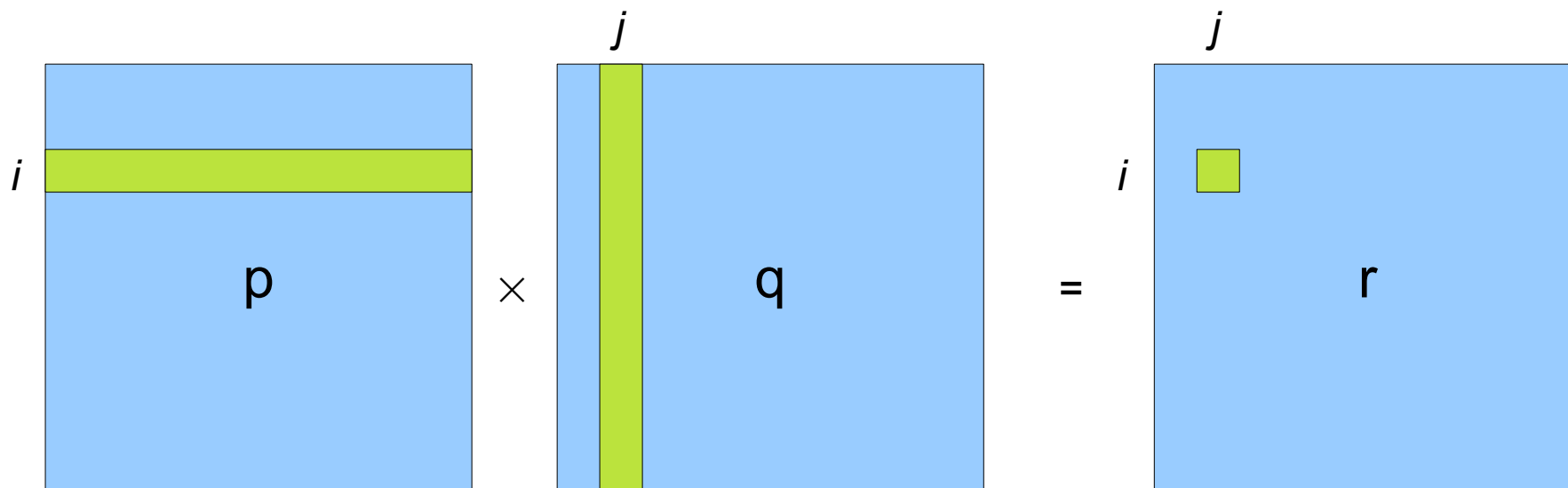
- **Decompose** the problem in sub-problems
- **Distribute** sub-problems to the available execution units
- **Solve** sub-problems
 - **Cooperate** to solve sub-problems, if necessary
- **Combine** the partial results
- **Goals**
 - Reduce the wall-clock time
 - Balance the workload across execution units
 - Reduce communication and synchronization overhead

A case study in dense matrix-matrix multiplication

<https://github.com/mmarzolla/matmul-test>

Credits: Performance Engineering of Software Systems, MIT OpenCourseWare
<https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/>

Matmul



$$r_{ij} = \sum_{k=0}^{n-1} p_{ik} q_{kj}$$

Hardware

Feature	Value
Processor type	Intel(R) Core(TM) i9-12900F
Clock frequency	5.1 GHz
Processor chips	1
Processing cores	8 P-core + 8 E-core
Hyperthreading	2 way (→ 16 virtual P-cores)
Floating-point unit	614 Gflops
L1-icache	640 KB
L1-dcache	768 KB
L2-cache	14 MB
L3-cache	30 MB
DRAM	64 GB
OS	Ubuntu Linux 24.04
Python	3.12.3
gcc	Ubuntu 13.3.0-6ubuntu2~24.04
Java	OpenJDK 21.0.10 2026-01-20



Version 1: Python

```
# matmul-test.py
import sys, random

n = 4096

p = [[random.random() for row in range(n)]
      for col in range(n)]
q = [[random.random() for row in range(n)]
      for col in range(n)]
r = [[0 for row in range(n)]
      for col in range(n)]

start = time()
for i in range(n):
    for j in range(n):
        for k in range(n):
            r[i][j] += p[i][k] * q[k][j]
elapsed = time() - start
print(elapsed)
```

Version 1: Python

```
# matmul-test.py
import sys, random

n = 4096

p = [[random.random() for row in range(n)]
      for col in range(n)]
q = [[random.random() for row in range(n)]
      for col in range(n)]
r = [[0 for row in range(n)]
      for col in range(n)]

start = time()
for i in range(n):
    for j in range(n):
        for k in range(n):
            r[i][j] += p[i][k] * q[k][j]
elapsed = time() - start
print(elapsed)
```

Running time
= **6711** seconds
= **1.8** hours

$2n^3$ floating-point
operations in
6711
≈ 20 MFLOPS

Peak **≈ 614**
GFLOPS

Python gets
0.00003% of
peak

Version 2: Java

```
import java.util.Random;
import java.io.*;
public class matmul_test {
    final static int n = 4096;
    static float[][] p = new float[n][n];
    static float[][] q = new float[n][n];
    static float[][] r = new float[n][n];
    public static void main(String[] args) {
        Random rnd = new Random();
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                p[i][j] = rnd.nextFloat();
                q[i][j] = rnd.nextFloat();
                r[i][j] = 0;
            }
        }
        final long start = System.nanoTime();
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                for (int k=0; k<n; k++)
                    r[i][j] += p[i][k] * q[k][j];
        final long stop = System.nanoTime();
        System.out.println((stop - start)*1e-9);
    }
}
```

Version 2: Java

```
import java.util.Random;
import java.io.*;
public class matmul_test {
    final static int n = 4096;
    static float[][] p = new float[n][n];
    static float[][] q = new float[n][n];
    static float[][] r = new float[n][n];
    public static void main(String[] args) {
        Random rnd = new Random();
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                p[i][j] = rnd.nextFloat();
                q[i][j] = rnd.nextFloat();
                r[i][j] = 0;
            }
        }
        final long start = System.nanoTime();
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                for (int k=0; k<n; k++)
                    r[i][j] += p[i][k] * q[k][j];
        final long stop = System.nanoTime();
        System.out.println((stop - start)*1e-9);
    }
}
```

Running time
= **322** seconds
= **5.4** minutes

21 times faster
than Python

Version 3: C

```
void matmul(const float * restrict p,
            const float * restrict q,
            float * restrict r, int n)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            r[i*n + j] = 0;

    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            for (int k=0; k<n; k++)
                r[i*n + j] += p[i*n + k] * q[k*n + j];
}
```

Version 3: C

```
void matmul(const float * restrict p,
            const float * restrict q,
            float * restrict r, int n)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            r[i*n + j] = 0;

    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            for (int k=0; k<n; k++)
                r[i*n + j] += p[i*n + k] * q[k*n + j];
}
```

Running time
= **488** seconds
= **8.1** minutes

14 times faster
than Python

~1.5 times slower
than Java

Note: compiled without
any optimization flag -Ox

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76

Matmul 4096 × 4096

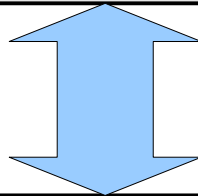
	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09

- Adding the `-O2` flag to gcc reduces the execution time by 30%
 - Still slower than Java!

Reordering loops

- In this particular examples, the three nested loops can be permuted and still produce the same result

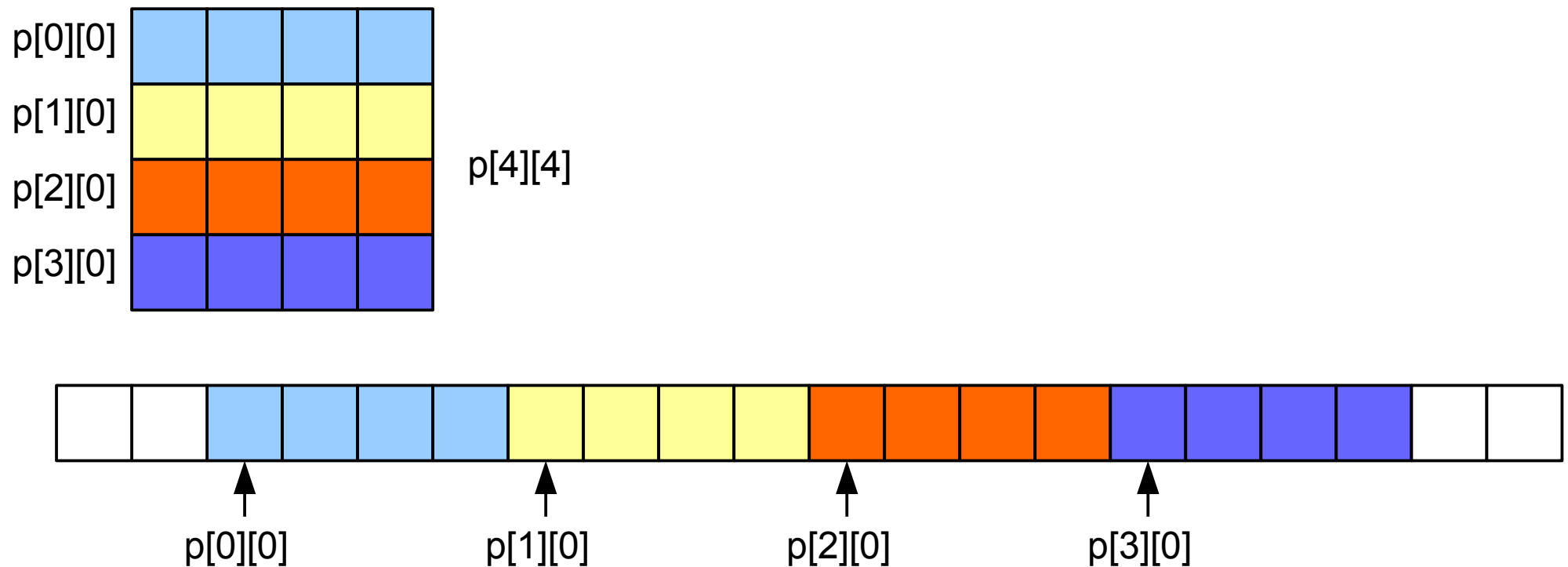
```
for (int i=0; i<n; i++)  
  for (int j=0; j<n; j++)  
    for (int k=0; k<n; k++)  
      r[i*n+j] += p[i*n+k] * q[k*n+j];
```



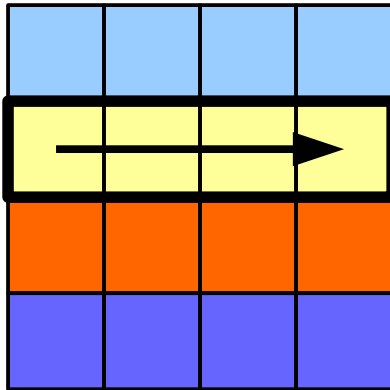
```
for (int k=0; k<n; k++)  
  for (int i=0; i<n; i++)  
    for (int j=0; j<n; j++)  
      r[i*n+j] += p[i*n+k] * q[k*n+j];
```

Matrix representation

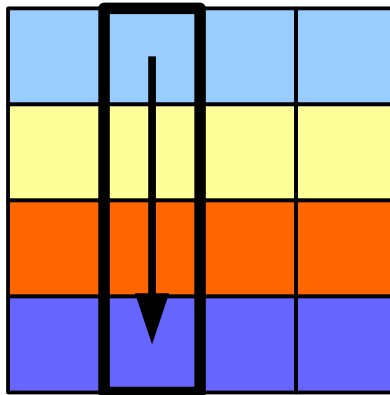
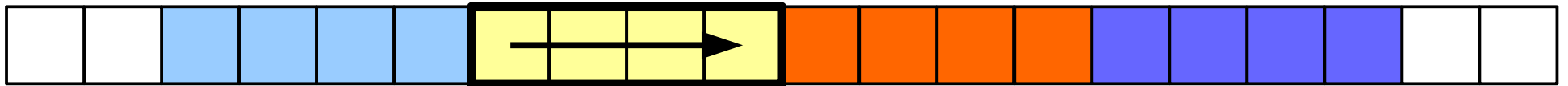
- Matrices in C are stored in *row-major order*
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory



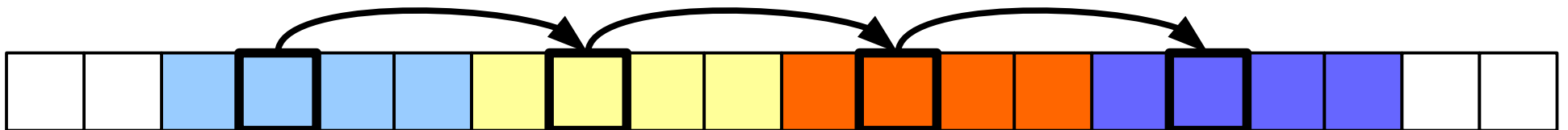
Row-wise vs column-wise access



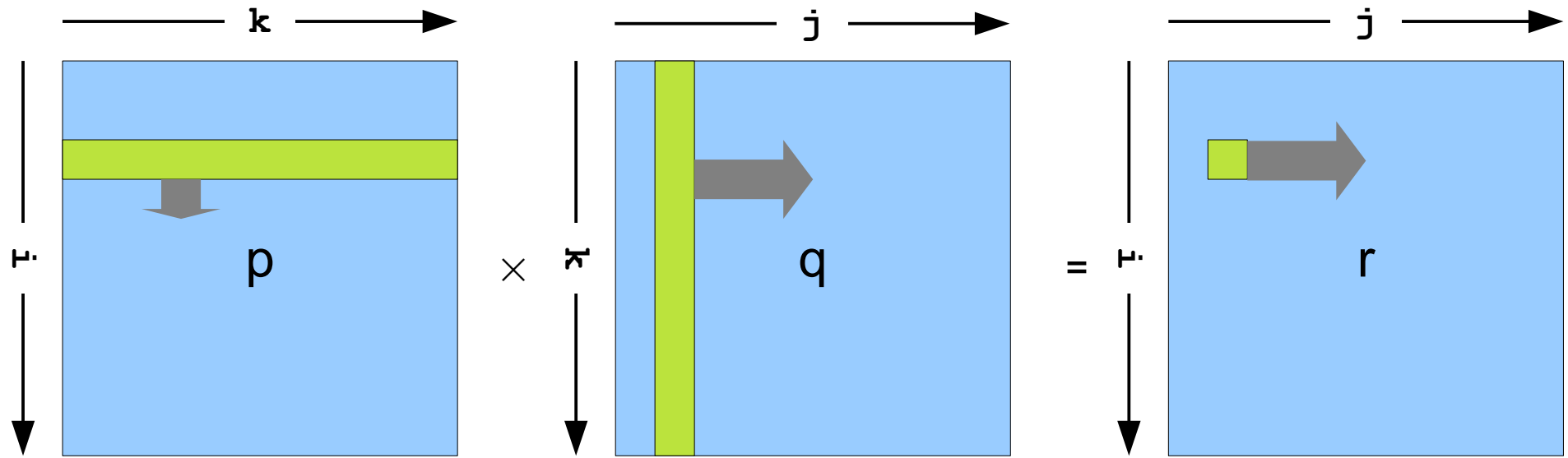
Row-wise access is OK: the data is contiguous in memory, so the cache helps (spatial locality)



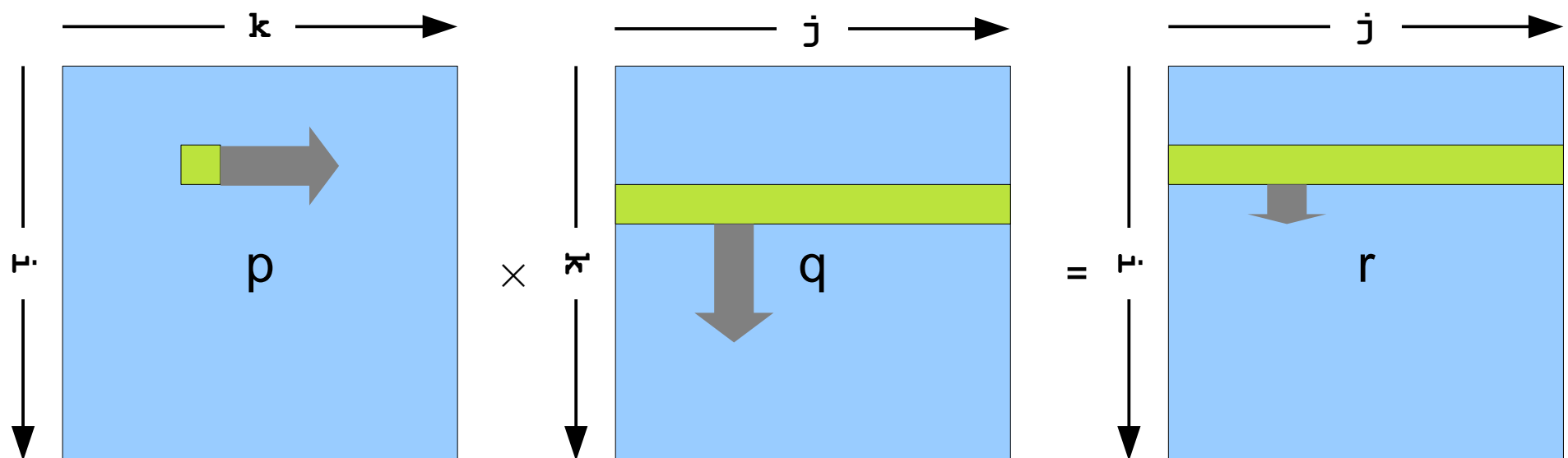
Column-wise access is **NOT OK**: the accessed elements are not contiguous in memory (*strided* access) so the cache does NOT help



i, j, k → 334s



i, k, j → 21s



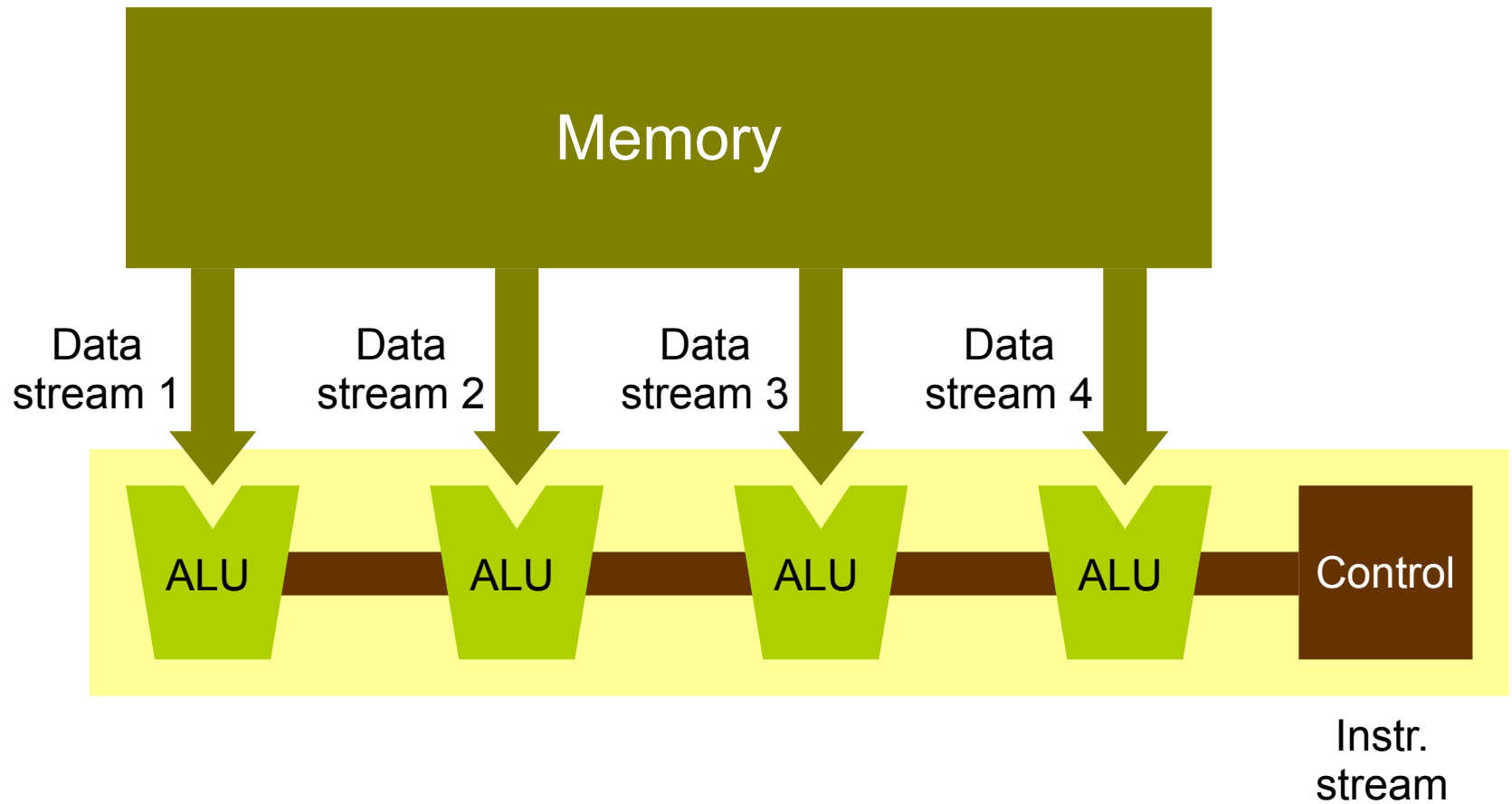
Reordering loops

Permutation	Time (s)	GFLOPS
i,j,k	334.08	0.41
i,k,j	20.54	6.69
j,i,k	182.33	0.75
j,k,i	593.29	0.23
k,i,j	21.61	6.36
k,j,i	542.72	0.25

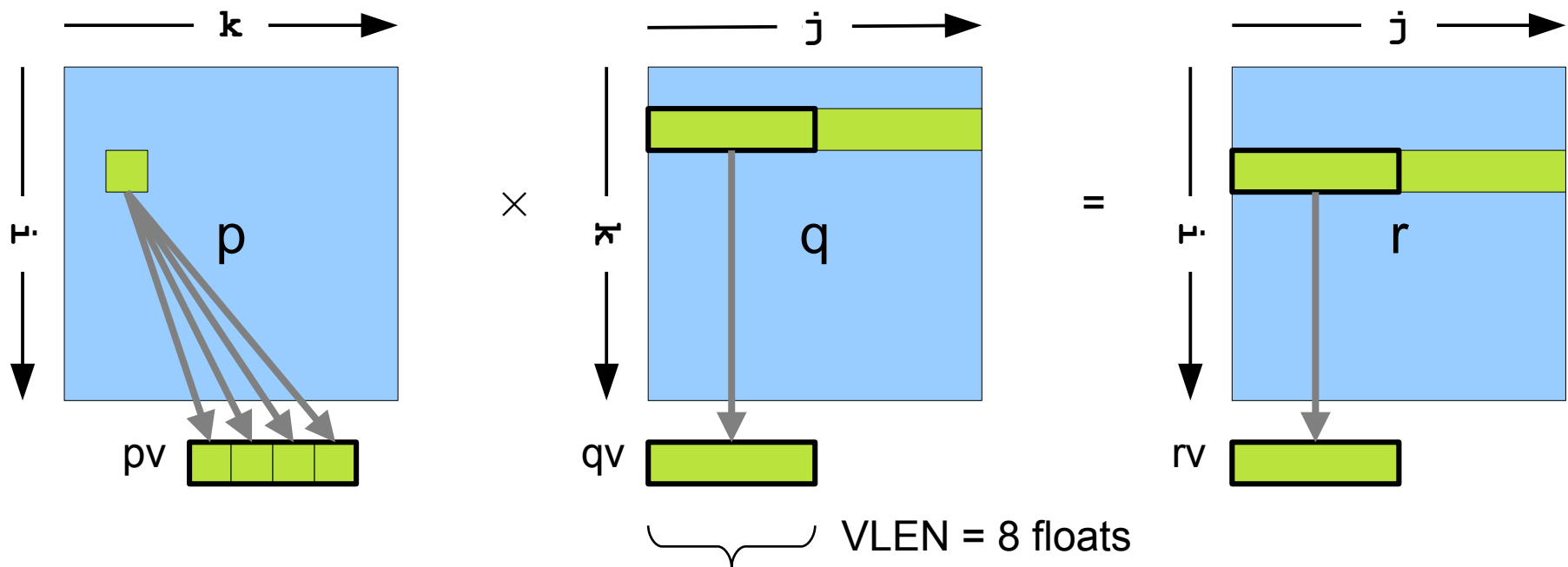
Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09
+ Loop reordering	20.54	6.69	1.09%	326.77

SIMD instructions



SIMD instructions



```

pv = _mm256_set1_ps(p[i*n + k]);           // set all lanes of pv to pik
rv = _mm256_load_ps( &r[i*n + j] );       // load rv
qv = _mm256_load_ps( &q[k*n + j] );       // load qv
rv = _mm256_fmadd_ps(pv, qv, rv);          // rv = pv * qv + rv
_mm256_store_ps( &r[i*n + j], rv );       // store rv
    
```

```

void matmul_ikj_simd( const float * restrict p, const float
* restrict q, float * restrict r, int n )
{
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j += VLEN) {
            _mm256_store_ps( &r[i*n + j],
                            _mm256_setzero_ps() );
        }
    }

    for (int i=0; i<n; i++) {
        for (int k=0; k<n; k++) {
            const __m256 pv = _mm256_broadcast_ss(&p[i*n + k]);
            for (int j=0; j<n; j += VLEN) {
                __m256 rv = _mm256_load_ps( &r[i*n + j] );
                const __m256 qv = _mm256_load_ps( &q[k*n + j] );
                rv = _mm256_fmadd_ps(pv, qv, rv);
                _mm256_store_ps( &r[i*n + j], rv );
            }
        }
    }
}

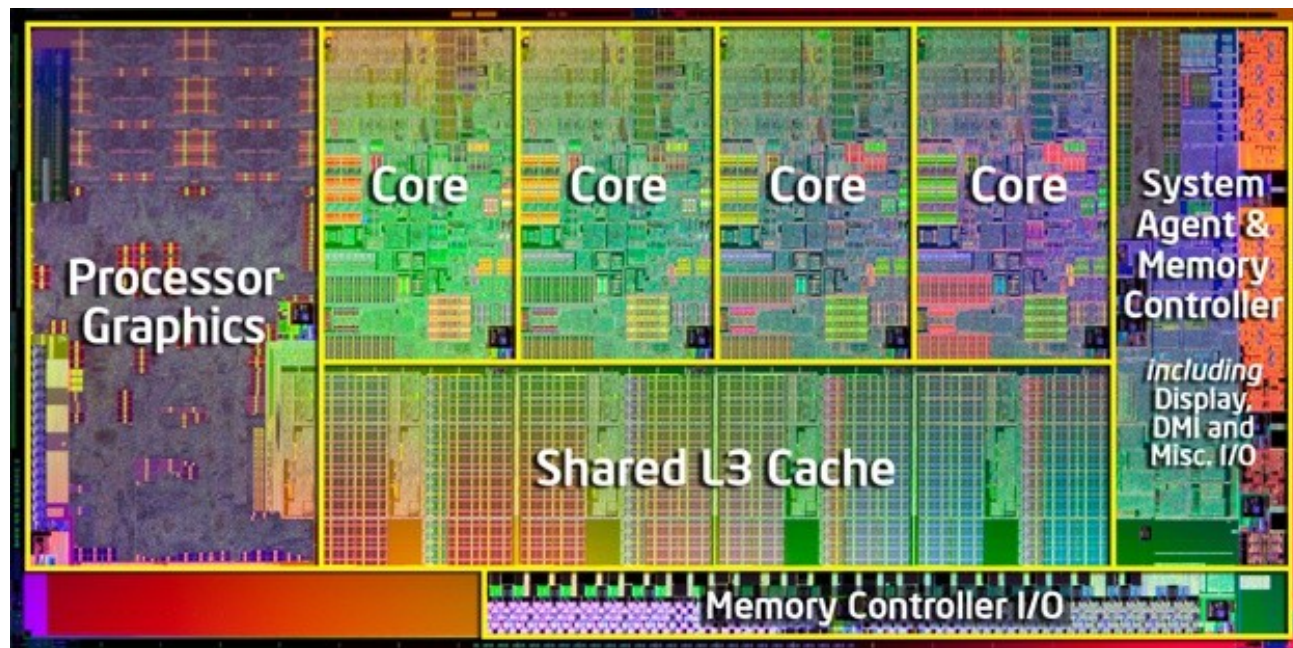
```

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09
+ Loop reordering	20.54	6.69	1.09%	326.77
+ auto SIMD	10.15	13.54	2.21%	661.33
+ manual SIMD	12.48	11.01	1.79%	537.64

Multicore parallelism

- Modern processors have multiple independent execution units (“cores”)
 - This machine has 8 of them, each one seen by the OS as two “virtual” cores (Intel *HyperThreading*)
 - Virtual cores share some physical resources and are less powerful than “true” physical cores



```

void matmul_ikj_simd_omp( const float * restrict p, const
float * restrict q, float * restrict r, int n) {
#pragma omp parallel default(none) shared(p,q,r,n)
{
#pragma omp for
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j += VLEN) {
            _mm256_store_ps( &r[i*n + j],
                            _mm256_setzero_ps() );
        }
    }

#pragma omp for
    for (int i=0; i<n; i++) {
        for (int k=0; k<n; k++) {
            const __m256 pv = _mm256_broadcast_ss(&p[i*n + k]);
            for (int j=0; j<n; j += VLEN) {
                __m256 rv = _mm256_load_ps( &r[i*n + j] );
                const __m256 qv = _mm256_load_ps( &q[k*n + j] );
                rv = _mm256_fmadd_ps(pv, qv, rv);
                _mm256_store_ps( &r[i*n + j], rv );
            }
        }
    }
}
}

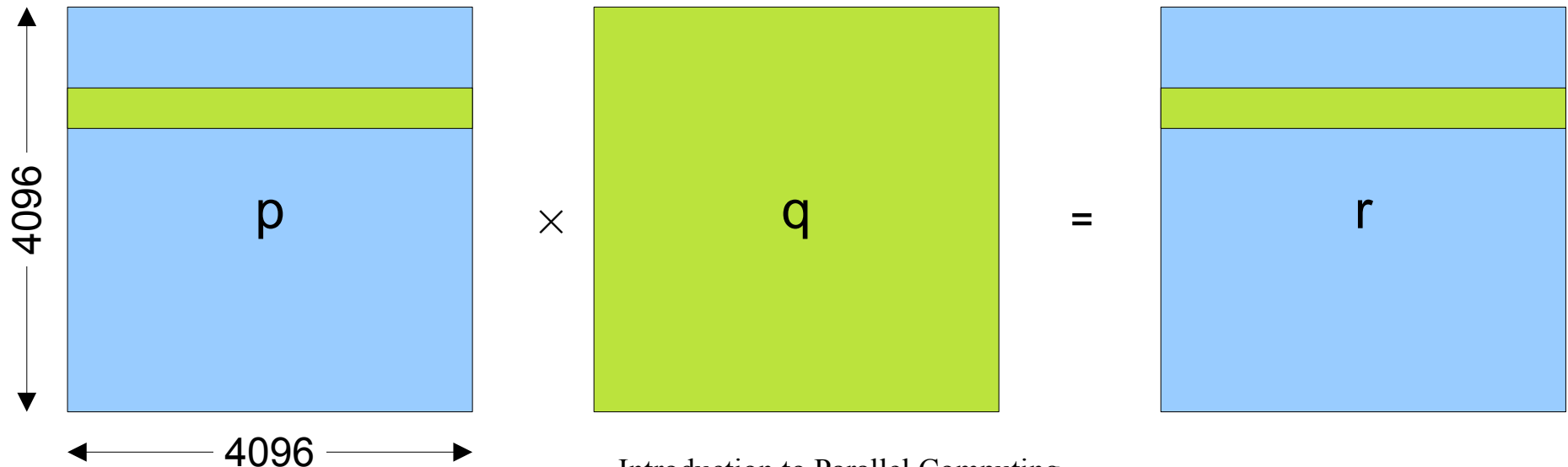
```

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09
+ Loop reordering	20.54	6.69	1.09%	326.77
+ auto SIMD	10.15	13.54	2.21%	661.33
+ manual SIMD	12.48	11.01	1.79%	537.64
+ manual SIMD + OpenMP	1.24	110.57	18.01%	5399.72

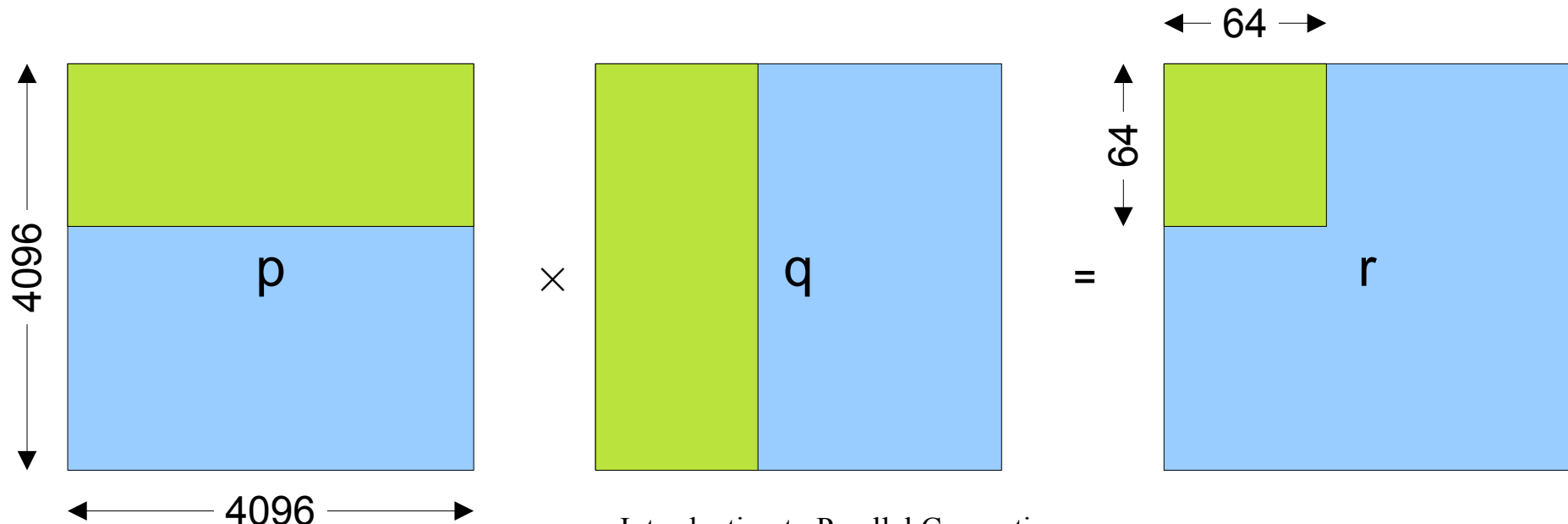
Tiling

- How many memory accesses are required to compute a row of r ?
 - 4096 writes of r
 - 4096 reads of p
 - $4096 \times 4096 = 16777216$ reads of q
 - 16785408 word transfers total



Tiling

- Instead of computing a row of r (4096 elements), let us compute a 64×64 block (still 4096 elements)
 - $64 \times 64 = 4096$ writes of r
 - $64 \times 4096 = 262144$ reads of p
 - $4096 \times 64 = 262144$ reads of q
 - 528384 word transfers

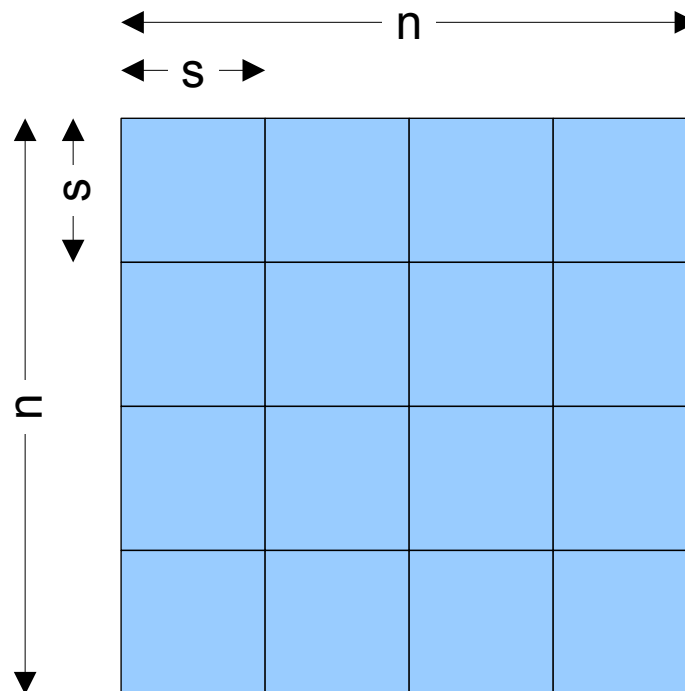


Tiled matrix-matrix multiply

```
for (int i=0; i<n; i += s)
  for (int j=0; j<n; j += s)
    for (int k=0; k<n; k += s)
      for (int ii=0; ii<s; ii++)
        for (int jj=0; jj<s; jj++)
          for (int kk=0; kk<s; kk++)
            r[i+ii][j+jj] += p[i+ii][k+kk] * q[k+kk][j+jj];
```

} Loop over tiles

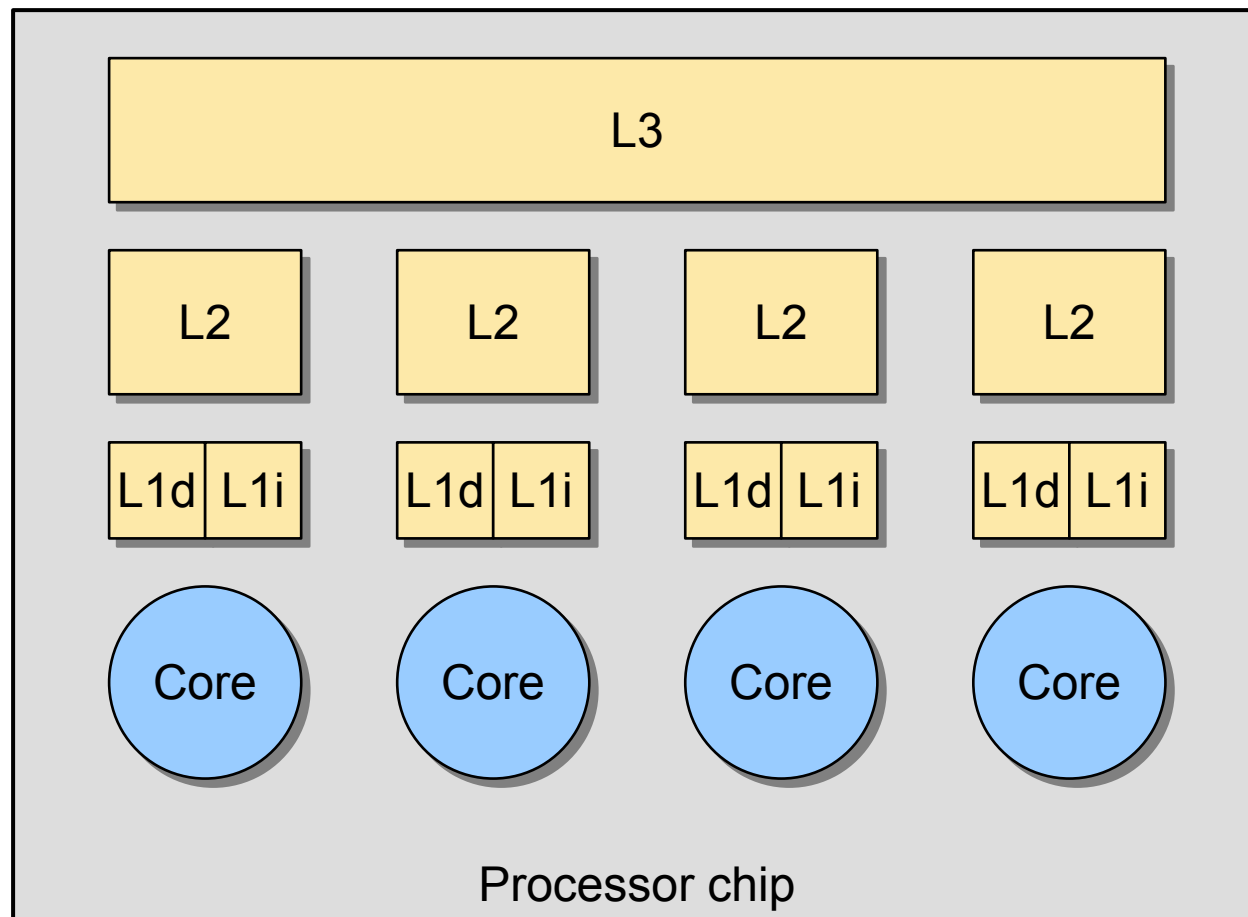
} Loop within tiles



What is the optimal value of s ?

Multilevel caches

- Modern processors have multiple levels of caches

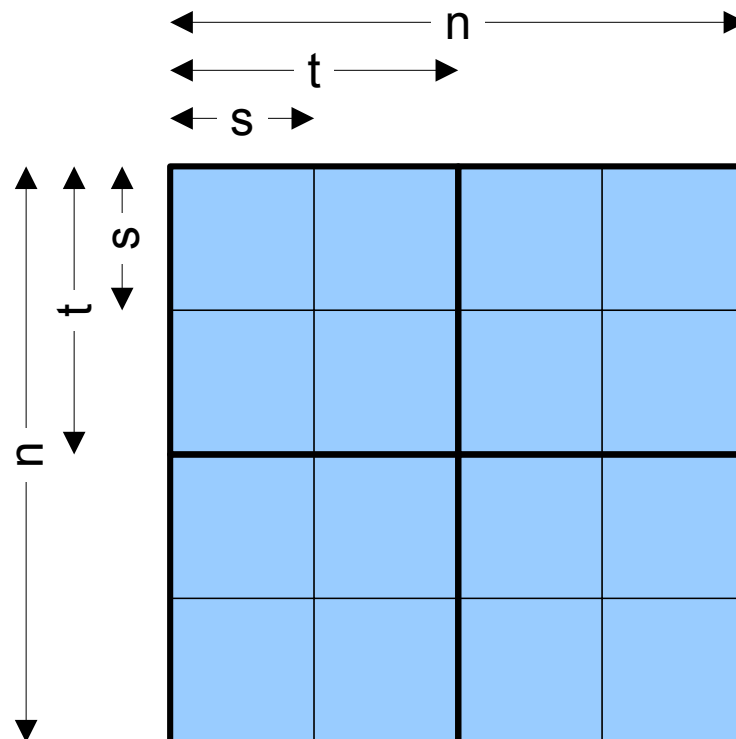


```

for (int i=0; i<n; i += t)
  for (int j=0; j<n; j += t)
    for (int k=0; k<n; k += t)
      for (int ii=0; ii<t; ii += s)
        for (int jj=0; jj<t; jj += s)
          for (int kk=0; kk<t; kk += s)
            for (int iii=0; iii<s; iii++)
              for (int jjj=0; jjj<s; jjj++)
                for (int kkk=0; kkk<s; kkk++)
                  r[i+ii+iii][j+jj+jjj] += p[i+ii+iii][k+kk+kkk] *
                    q[k+kk+kkk][j+jj+jjj];

```

} Loop over big tiles
 } Loop over smaller tiles within big tile
 } Loop within tile



What are the optimal values of s and t ?

Parallel divide-and-conquer

- Improves data locality and cache usage
- Agnostic with respect to the number of levels of caching and their sizes
- Idea
 - Decompose A and B into four smaller matrices
 - Compute $A \times B$ as sums of eight products
 - Compute the products recursively

$A_{0,0}$	$A_{0,1}$
$A_{1,0}$	$A_{1,1}$

 $\times$

$B_{0,0}$	$B_{0,1}$
$B_{1,0}$	$B_{1,1}$

 $=$

$A_{0,0}B_{0,0}$	$A_{0,0}B_{0,1}$
$A_{1,0}B_{0,0}$	$A_{1,0}B_{0,1}$

 $+$

$A_{0,1}B_{1,0}$	$A_{0,1}B_{1,1}$
$A_{1,1}B_{1,0}$	$A_{1,1}B_{1,1}$

```

void mm_rec( float * restrict C, const float * restrict A, const
float * restrict B, int n, int b) {
    const int THR = 64; /* tuning param. */
    if (b <= THR) {
        mm_base(C, A, B, n, b);
    } else {
#define SUB(M,ro,co) ((M) + ((ro)*n+(co)) * ((b)/2))
#pragma omp task
        mm_rec(SUB(C,0,0), SUB(A,0,0), SUB(B,0,0), n, b/2);
#pragma omp task
        mm_rec(SUB(C,0,1), SUB(A,0,0), SUB(B,0,1), n, b/2);
#pragma omp task
        mm_rec(SUB(C,1,0), SUB(A,1,0), SUB(B,0,0), n, b/2);
#pragma omp task
        mm_rec(SUB(C,1,1), SUB(A,1,0), SUB(B,0,1), n, b/2);
#pragma omp taskwait
#pragma omp task
        mm_rec(SUB(C,0,0), SUB(A,0,1), SUB(B,1,0), n, b/2);
#pragma omp task
        mm_rec(SUB(C,0,1), SUB(A,0,1), SUB(B,1,1), n, b/2);
#pragma omp task
        mm_rec(SUB(C,1,0), SUB(A,1,1), SUB(B,1,0), n, b/2);
#pragma omp task
        mm_rec(SUB(C,1,1), SUB(A,1,1), SUB(B,1,1), n, b/2);
#pragma omp taskwait
    }
}

```

- The matrix size n must be a power of two.
- Start the recursion with `mm_rec(r, p, q, n, n)`

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	6711.85	0.02	0.00%	1.00
Java	322.02	0.43	0.07%	20.84
C	487.86	0.28	0.05%	13.76
+ -O2 flag	334.08	0.41	0.07%	20.09
+ Loop reordering	20.54	6.69	1.09%	326.77
+ auto SIMD	10.15	13.54	2.21%	661.33
+ manual SIMD	12.48	11.01	1.79%	537.64
+ manual SIMD + OpenMP	1.24	110.57	18.01%	5399.72
Parallel divide-and-conquer	0.45	308.85	50.30%	15082.80